

SÉANCE 4

Recherche de fichiers et outils de traitement de texte

Expressions régulières et caractères spéciaux sous Linux

Ouafae EL AISSAOUI

ouafae.elaissoui@usmba.ac.ma



Contexte : utilisation des commandes Linux

Contexte : utilisation des commandes Linux

Caractères spéciaux

Interprétés directement par le shell (Bash)

Fonction : contrôler l'exécution des commandes

- Redirection (> < >>)
- Chaînage (| && ||)
- Substitution (\$() ^")
- Génération de noms (? * {})
- Lancement en arrière-plan (&)

Simplifie l'écriture des commandes • Automatisation de tâches

Expressions régulières

Utilisées dans des commandes de texte (grep, sed, awk, etc.)

Fonction : rechercher et filtrer du texte

- Motifs simples : abc ^mot fin\$
- Classes de caractères : [az] [bc]
- Quantificateurs : * + ? {n,m}
- Groupes et alternatives : () |
- Extraction d'informations précises

Extraction d'informations précises • Analyse de texte ou de journaux



A. Les caractères spéciaux

Les caractères spéciaux du shell sont souvent utilisés dans des scripts ou des commandes pour :



Rechercher des fichiers et des répertoires (Globbing)



Redirection de l'entrée/sortie



Exécution conditionnelle

Les caractères spéciaux sont interprétés directement par **l'interpréteur du shell** (comme Bash).



1. Le Globbing

Correspondance de motifs (globbing) : le globbing est une technique utilisée par les systèmes Linux pour rechercher des fichiers et des répertoires en utilisant des caractères spéciaux.

Caractère spécial	Utilisation	Exemple	Description
*	Correspond à n'importe quel nombre de caractères	ls *.txt	Correspond à tous les fichiers se terminant par .txt.
?	Correspond à un seul caractère	ls file?.txt	Correspond à des fichiers comme file1.txt, fileA.txt, etc.
[]	Correspond à n'importe quel caractère parmi ceux listés	ls file[1-3].txt	Correspond à file1.txt, file2.txt, file3.txt.
{ }	Expansion de séquence	echo file{1,2,3}	Génère file1 file2 file3.



Les crochets [] : syntaxe d'utilisation

Syntaxe	Utilisation	Description	Contexte
file[1-3]	Plage ou ensemble de caractères	Correspond à n'importe quel fichier nommé file1, file2, ou file3.	Globbering / RegEx
file[!1-3]	Exclusion de caractères	Correspond à n'importe quel fichier qui n'est pas file1, file2, ou file3.	Globbering / RegEx
file[a-c]	Plage de lettres	Correspond à des fichiers nommés filea, fileb, ou filec.	Globbering / RegEx
file[aed]	Ensemble de caractères	Affiche les fichiers nommés filea, filee, et filed s'ils existent.	Globbering / RegEx
file[a-f0-3]	Combinaison de plusieurs plages	Affiche les fichiers nommés filea, fileb, filec, file0, file1, file2.	Globbering / RegEx



Utilisation des crochets [] dans le Globbing

Dans le contexte du globbing, les crochets peuvent être utilisés pour rechercher des fichiers ou des répertoires en utilisant des motifs simples.

Commande	Utilisation	Exemple	Description
ls	Listing de fichiers correspondant au modèle	ls file[1-3].txt	Affiche les fichiers file1.txt, file2.txt, et file3.txt s'ils existent.
rm	Suppression de fichiers correspondant au modèle	rm file[1-3].txt	Supprime les fichiers file1.txt, file2.txt, et file3.txt s'ils existent.
find	Recherche de fichiers correspondant au modèle	find . -name 'file[1-3].txt'	Recherche les fichiers file1.txt, file2.txt, et file3.txt dans le répertoire courant.
cp	Copie de plusieurs fichiers	cp file[12].txt /backup/	Copie les fichiers file1.txt et file2.txt dans le répertoire /backup/.
mv	Déplacement de plusieurs fichiers	mv file[ab].txt /archive/	Déplace les fichiers filea.txt et fileb.txt dans le répertoire /archive/.



Utilisation des accolades {} dans le Globbing

Caractère spécial	Utilisation	Exemple	Description
{}	Expansion de liste	file{1,2,3}.txt	Correspond aux fichiers file1.txt, file2.txt, et file3.txt.
{}	Expansion de séquence numérique	file{1..3}.txt	Correspond aux fichiers file1.txt, file2.txt, et file3.txt.
{}	Expansion de séquence avec pas	file{1..5..2}.txt	Correspond aux fichiers file1.txt, file3.txt, et file5.txt (avec un pas de 2).
{}	Expansion de séquence alphabétique	file{a..c}.txt	Correspond aux fichiers filea.txt, fileb.txt, et filec.txt.
{}	Expansion combinatoire	{file1,file2}.{txt,log}	Correspond aux fichiers file1.txt, file1.log, file2.txt, et file2.log.
{}	Expansion combinatoire complexe	{doc,report}_{1,2}.{pdf,docx}	Correspond aux fichiers doc_1.pdf, doc_1.docx, doc_2.pdf, doc_2.docx, report_1.pdf, etc.



Les accolades {} : commandes de recherche

Dans le contexte du globbing, les accolades peuvent être utilisées pour rechercher des fichiers ou des répertoires en utilisant des motifs simples, ainsi que pour sélectionner plusieurs fichiers ou répertoires en une seule commande.

Commande	Utilisation	Exemple	Description
ls	Listing de fichiers avec expansion	ls file{1,2,3}.txt	Affiche les fichiers file1.txt, file2.txt, et file3.txt s'ils existent.
find	Recherche avec plusieurs critères	find . -name '{file1,file2}.txt'	Recherche les fichiers file1.txt ou file2.txt dans le répertoire courant et ses sous-répertoires.
cp	Copie de plusieurs fichiers	cp file{1,2}.txt /backup/	Copie les fichiers file1.txt et file2.txt dans le répertoire /backup/.
mv	Déplacement de plusieurs fichiers	mv file{a,b}.txt /archive/	Déplace les fichiers filea.txt et fileb.txt dans le répertoire /archive/.
rm	Suppression de plusieurs fichiers	rm file{1..3}.txt	Supprime les fichiers file1.txt, file2.txt, et file3.txt.

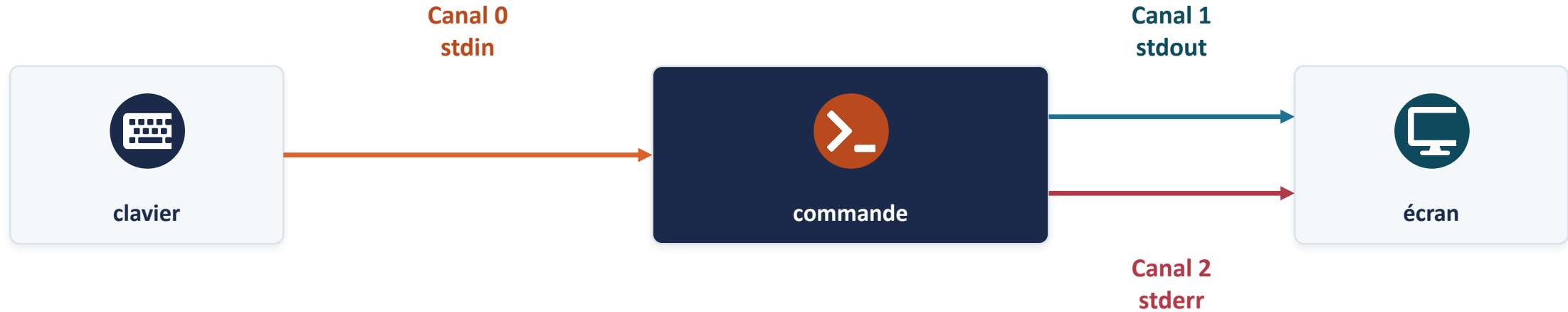


Les accolades {} : commandes de création

Commande	Utilisation	Exemple	Description
mkdir	Création de répertoires multiples	<code>mkdir project_{1,2,3}</code>	Crée les répertoires <code>project_1</code> , <code>project_2</code> , et <code>project_3</code> .
touch	Création de plusieurs fichiers	<code>touch report_{a,b,c}.txt</code>	Crée les fichiers <code>report_a.txt</code> , <code>report_b.txt</code> , et <code>report_c.txt</code> .
echo	Affichage de combinaisons	<code>echo {apple,banana}-{red,green}</code>	Affiche les combinaisons : <code>apple-red</code> , <code>apple-green</code> , <code>banana-red</code> , <code>banana-green</code> .



2. Les flux d'entrée/sortie (< ; > ; >> ; |)



Dans un système Linux, chaque commande que vous exécutez interagit avec trois flux standard :

- L'entrée standard (stdin) est la source des données d'entrée. Par défaut, stdin est tout texte saisi au clavier.
- La sortie standard (stdout) est le résultat de la commande. Par défaut, elle est affichée à l'écran.
- L'erreur standard (stderr) est le message d'erreur (le cas échéant) produit par les commandes. Par défaut, stderr est également affiché à l'écran. Son ID de flux est 2.



Redirection STDOUT : > ou >>



Rediriger stdout (sortie standard) de la commande vers un fichier

Créer / écrire / modifier un fichier en sortie

Exemple

```
echo "Hello" > file.txt      # écrase file.txt et écrit Hello
echo "Hello" >> file.txt     # ajoute Hello à la fin de file.txt
```



Redirection STDIN <



fichier



Commande

Rediriger un fichier comme entrée standard (STDIN) vers une commande

Exemple

```
wc -l < fichier.txt      # compte les lignes de fichier.txt utilisé comme entrée
```



La redirection pipe |



Rediriger la sortie standard (STDOUT) d'une commande vers l'entrée standard (STDIN) d'une autre commande.

Exemple

```
ls | grep "pattern" # liste les fichiers, puis filtre ceux contenant "pattern"  
ls | wc -l          # liste les fichiers, puis compte le nombre de lignes obtenues
```



Récapitulatif : les flux d'entrée et de sortie

Caractère spécial	Utilisation	Exemple	Description
>	Redirige la sortie standard vers un fichier	echo "Hello" > file.txt	Écrit Hello dans file.txt, écrasant le contenu existant.
>>	Redirige la sortie standard vers un fichier (ajout)	echo "Hello" >> file.txt	Ajoute Hello à la fin de file.txt sans écraser le contenu existant.
<	Redirige l'entrée standard à partir d'un fichier	wc -l < file.txt	Compte les lignes de file.txt en utilisant le contenu du fichier comme entrée.
(pipe)	Dirige la sortie d'une commande vers l'entrée d'une autre commande.	ls grep "pattern"	Liste les fichiers et filtre ceux correspondant à "pattern".



3. Exécution conditionnelle

Caractère spécial	Utilisation	Exemple	Description
&& (AND logique)	Exécute la deuxième commande si la première réussit.	<code>mkdir dir && cd dir</code>	Crée le dossier dir et s'y déplace uniquement si la création a réussi.
;	Séparateur pour exécuter plusieurs commandes	<code>echo "Bonjour" ; ls -l ; date</code>	Exécute les commandes de manière séquentielle.
 (OR logique)	Exécute la deuxième commande uniquement si la première échoue.	<code>mkdir mon_dossier echo "La création du dossier a échoué."</code>	Si la création du dossier réussit, la commande suivante ne s'exécute pas ; en cas d'échec, echo affiche le message.



3. Exécution conditionnelle et échappement des caractères

Caractère spécial	Utilisation	Exemple	Description
()	Crée un sous-shell pour exécuter des commandes	(cd ~; ls)	Change temporairement de répertoire pour lister les fichiers, sans changer le répertoire courant du shell parent.
` `	Exécuter une commande dans une commande (exécution de commandes imbriquées)	echo `date`	Remplace les guillemets inversés par le résultat de la commande date, affichant la date courante.
\	Échappe un caractère spécial pour qu'il soit traité littéralement	touch *.txt	touch *.txt ne crée pas de fichier, car * sert à sélectionner des fichiers existants. Il faut échapper son comportement avec \.



B. Les expressions régulières

Les expressions régulières sont des motifs utilisés pour rechercher ou manipuler du texte dans des commandes comme `grep`, `sed`, et `awk`. Contrairement aux caractères spéciaux, qui sont directement interprétés par le shell, ces outils disposent de leurs propres moteurs d'interprétation des expressions régulières, permettant ainsi de rechercher des motifs complexes dans du texte.



Caractères spéciaux

Interprétés directement par le shell (Bash), avant l'exécution de la commande.



Expressions régulières

Interprétées par le moteur interne de l'outil (`grep`, `sed`, `awk`...) qui traite le texte.



Manipulation de texte avec sed, grep, awk...

Expression régulière	Description	Exemple	Correspondances
.	Correspond à n'importe quel caractère (sauf le saut de ligne).	a.c	Correspond à "abc", "a9c", "a#c", etc.
*	Correspond à zéro ou plusieurs occurrences du caractère précédent.	a*b	Correspond à "b", "ab", "aab", "aaab", etc.
+	Correspond à une ou plusieurs occurrences du caractère précédent.	a+b	Correspond à "ab", "aab", "aaab", etc. (mais pas "b").
?	Correspond à zéro ou une occurrence du caractère précédent.	colou?r	Correspond à "color" et "colour".
^	Indique le début d'une ligne.	^Hello	Correspond aux lignes commençant par "Hello".
\$	Indique la fin d'une ligne.	world\$	Correspond aux lignes se terminant par "world".
[]	Correspond à un seul caractère parmi ceux spécifiés dans les crochets.	[aeiou]	Correspond à n'importe quelle voyelle.
[^]	Correspond à un seul caractère qui n'est pas parmi ceux spécifiés.	[^0-9]	Correspond à tout caractère qui n'est pas un chiffre.
()	Groupe une expression pour l'appliquer à des opérateurs ou pour capturer.	(abc)+	Correspond à "abc", "abcabc", etc.
	Opérateur logique OU, correspond à l'une des alternatives.	cat dog	Correspond à "cat" ou "dog".



Les crochets [] dans les expressions régulières

Syntaxe	Utilisation	Description et exemple d'usage avec egrep	Contexte
[0-9]	Plage de chiffres	Correspond à n'importe quel chiffre de 0 à 9. Exemple : egrep "[0-9]" fichier.txt → affiche les lignes contenant au moins un chiffre.	Expression régulière
[^0-9]	Exclusion de chiffres (un caractère)	Correspond à tout caractère qui n'est pas un chiffre (lettre, espace, symbole...).	Expression régulière
-v '[0-9]'	Inversion du motif (option -v)	Affiche les lignes qui ne contiennent pas le motif indiqué. Exemple : egrep -v "[0-9]" fichier.txt → affiche les lignes sans chiffres.	Commande egrep (filtrage négatif)
[a-zA-Z]	Plage de lettres	Correspond à toute lettre majuscule ou minuscule. Exemple : egrep "[a-zA-Z]" fichier.txt → affiche les lignes contenant au moins une lettre.	Expression régulière



Les accolades {} dans les expressions régulières

Notation	Description	Exemples	Explication
<code>{n}</code>	Précisément n occurrences du caractère ou groupe précédent.	<code>a{3}</code>	Correspond à exactement trois caractères a consécutifs, soit aaa.
<code>{n,}</code>	Au moins n occurrences du caractère ou groupe précédent.	<code>a{2,}</code>	Correspond à deux caractères a ou plus, comme aa, aaa, etc.
<code>{n,m}</code>	Entre n et m occurrences du caractère ou groupe précédent.	<code>a{1,3}</code>	Correspond à une, deux ou trois occurrences du caractère a. Exemples valides : a, aa, aaa.
<code>{,m}</code>	De zéro à m occurrences du caractère ou groupe précédent.	<code>a{,3}</code>	Correspond à zéro, une, deux ou trois occurrences du caractère a.



La commande SED

Introduction à sed

La commande sed (Stream Editor) est utilisée pour effectuer des modifications sur du texte provenant de fichiers ou de flux d'entrée (comme une sortie de commande). Elle permet de rechercher, remplacer, insérer ou supprimer des lignes dans un fichier.

Syntaxe générale de sed

```
sed [options] 'commande' fichier
```

- sed : l'éditeur de flux.
- [options] : options pour contrôler le comportement de sed.
- 'commande' : la commande sed à exécuter (par exemple, substitution, suppression).
- fichier : le fichier sur lequel sed doit opérer.



Les options de la commande sed

`sed [options] 'commande' fichier`

Option	Description	Exemple
-e	Permet d'ajouter et d'exécuter plusieurs commandes sed dans une seule ligne.	<code>sed -e 's/a/b/' -e 's/c/d/' fichier.txt</code>
-f	Charge et exécute des commandes sed depuis un fichier contenant un script.	<code>sed -f script.sed fichier.txt</code>
-n	Supprime l'affichage automatique des lignes ; seules celles explicitement demandées sont affichées.	<code>sed -n '/motif/p' fichier.txt</code>
-i	Modifie le fichier directement, sans créer de fichier temporaire.	<code>sed -i 's/ancien/nouveau/g' fichier.txt</code>
-r	Active l'utilisation des expressions régulières étendues (modernes) sans avoir à échapper certains métacaractères.	<code>sed -r 's/motif1/motif2/' fichier.txt</code>
-s	Traite chaque fichier d'entrée séparément lorsqu'il y a plusieurs fichiers.	<code>sed -s '1,2d' fichier1.txt fichier2.txt</code>
-l	Définit la longueur maximale des lignes dans la sortie, utile pour les fichiers avec de longues lignes.	<code>sed -l 80 fichier.txt</code>
--help	Affiche l'aide et la liste des options disponibles pour la commande sed.	<code>sed --help</code>
--version	Affiche la version de sed installée sur le système.	<code>sed --version</code>



Les commandes dans la commande sed (1/2)

`sed [options] 'commande' fichier`

Commande sed	Description	Exemple
<code>s/motif/remplacement/</code>	Remplace la première occurrence du motif par une chaîne de remplacement.	<code>sed 's/chat/chien/' fichier.txt</code>
<code>s/motif/remplacement/g</code>	Remplace toutes les occurrences du motif par une chaîne de remplacement.	<code>sed 's/chat/chien/g' fichier.txt</code>
<code>Nd</code>	Supprime la ligne N.	<code>sed '3d' fichier.txt</code>
<code>/motif/d</code>	Supprime les lignes contenant le motif spécifié.	<code>sed '/Erreur/d' fichier.txt</code>
<code>Np</code>	Affiche uniquement la ligne N.	<code>sed -n '3p' fichier.txt</code>
<code>/motif/p</code>	Affiche uniquement les lignes qui contiennent le motif spécifié.	<code>sed -n '/Alice/p' fichier.txt</code>
<code>i texte</code>	Insère du texte avant une ligne spécifiée.	<code>sed '3i Bonjour tout le monde' fichier.txt</code>
<code>a texte</code>	Ajoute du texte après une ligne spécifiée.	<code>sed '\$a Fin du fichier' fichier.txt</code>
<code>c texte</code>	Remplace une ligne ou un ensemble de lignes par du texte.	<code>sed '3c Nouvelle ligne' fichier.txt</code>
<code>y/car1/car2/</code>	Remplace les caractères de car1 par ceux de car2 (équivalent à tr).	<code>sed 'y/abc/ABC/' fichier.txt</code>



Les commandes dans la commande sed (2/2)

`sed [options] 'commande' fichier`

Commande sed	Description	Exemple
q	Quitte sed après un certain nombre de lignes traitées.	sed '10q' fichier.txt
N	Ajoute la ligne suivante au pattern space (utilisé pour manipuler plusieurs lignes à la fois).	sed 'N; s/\n/ /' fichier.txt (joint les lignes deux par deux)
=	Affiche le numéro des lignes du fichier.	sed '=' fichier.txt



La commande egrep

egrep est une commande utilisée pour rechercher des motifs dans des fichiers en utilisant des expressions régulières étendues (ERE). C'est un outil très puissant pour filtrer des lignes de texte en fonction de motifs spécifiques.

Syntaxe

```
egrep [OPTIONS] PATTERN [FICHIER...]
```

- **PATTERN** : le motif que vous recherchez. Ce motif peut inclure des expressions régulières étendues.
- **FICHIER** : un ou plusieurs fichiers dans lesquels effectuer la recherche. Si aucun fichier n'est spécifié, egrep lit l'entrée standard.



La commande egrep : les options (1/2)

Option	Description	Exemple
-i	Ignore la casse lors de la recherche.	egrep -i "pattern" fichier.txt : affiche les lignes qui contiennent le motif « pattern » (ou une variante de casse, comme « Pattern », « PATTERN », etc.).
-v	Affiche les lignes qui ne correspondent pas au motif.	egrep -v "pattern" fichier.txt : affiche toutes les lignes de fichier.txt qui ne contiennent pas le motif « pattern ».
-c	Compte le nombre de lignes correspondantes.	egrep -c "pattern" fichier.txt : affiche le nombre total de lignes dans fichier.txt contenant le motif « pattern ».
-n	Affiche les numéros de ligne des correspondances.	egrep -n "pattern" fichier.txt : affiche les lignes contenant « pattern » avec leur numéro de ligne. Par exemple, si « pattern » est sur la ligne 5, le résultat sera « 5: [contenu de la ligne] ».
-r ou -R	Recherche réursive dans les sous-répertoires.	egrep -r "pattern" /chemin/vers/répertoire : recherche toutes les lignes contenant « pattern » dans les fichiers du répertoire spécifié et de ses sous-répertoires.



La commande egrep : les options (2/2)

Option	Description	Exemple
-l	Affiche uniquement les noms des fichiers contenant le motif.	<code>egrep -l "pattern" /chemin/vers/*.txt</code> : affiche uniquement les noms des fichiers .txt contenant le motif « pattern », sans afficher les lignes correspondantes.
-o	Affiche uniquement la partie de la ligne qui correspond au motif.	<code>egrep -o "pattern" fichier.txt</code> : affiche seulement les correspondances exactes du motif, sans le reste de la ligne.
--color	Met en surbrillance les correspondances dans la sortie.	<code>egrep --color "pattern" fichier.txt</code> : affiche les lignes contenant « pattern » avec les correspondances mises en surbrillance en couleur.
-E	Permet d'utiliser des expressions régulières étendues (équivalent à egrep).	<code>grep -E "pattern" fichier.txt</code> : recherche « pattern » en utilisant des expressions régulières étendues, fonctionnellement identique à <code>egrep "pattern" fichier.txt</code> .



La commande AWK

Introduction à awk

awk est un langage de programmation orienté texte utilisé pour traiter et analyser des données structurées ou non. Il est souvent utilisé pour parcourir des fichiers ligne par ligne, rechercher des motifs, extraire des colonnes spécifiques et effectuer des opérations simples.

```
awk 'condition  
    { action }'
```

condition

Une condition qui détermine quelles lignes sont traitées

action

Une action à effectuer sur les lignes sélectionnées

awk est une commande programmable : on peut écrire un petit programme pour définir précisément le traitement à effectuer, avec des conditions, des actions, des calculs et des formats d'affichage.



Les variables intégrées dans la commande awk

Les variables intégrées dans une commande awk servent à faciliter le traitement des données en fournissant des informations dynamiques sur l'état du fichier ou des lignes en cours de traitement.

awk [options] 'conditions {actions}' fichier

Variable	Description
\$0	Contient l'intégralité de la ligne en cours.
\$n	Contient la valeur du nième champ sur la ligne en cours (ex : \$1 pour le premier champ, \$2 pour le second).
NF	Contient le nombre total de champs sur la ligne en cours.
NR	Contient le numéro de la ligne actuelle (enregistrements).
FNR	Contient le numéro de la ligne actuelle dans le fichier en cours (utile lors du traitement de plusieurs fichiers).
FILENAME	Contient le nom du fichier en cours de traitement.



Les conditions dans la commande awk (1/2)

Dans la commande awk, les conditions sont utilisées dans la partie « condition » pour filtrer les lignes du fichier à traiter.

awk [options] 'conditions {actions}' fichier

Type de condition	Exemple
Sans condition	{print \$1}
Expression régulière (~, !~)	awk '\$0 ~ /Alice/' fichier.txt
Comparaison numérique (==, >, <)	awk '\$2 == 50' fichier.txt
Comparaison textuelle (==, !=)	awk '\$1 == "Alice"' fichier.txt



Les conditions dans la commande awk (2/2)

Dans la commande awk, les conditions sont utilisées dans la partie « condition » pour filtrer les lignes du fichier à traiter.

awk [options] 'conditions {actions}' fichier

Type de condition	Exemple
Opérateur logique AND (&&)	awk '\$2 > 50 && \$3 < 100' fichier.txt
Opérateur logique OR ()	awk '\$2 > 50 \$3 < 100' fichier.txt
Champs vides / non définis	awk '\$2 == ""' fichier.txt
Numéro de ligne (NR) — afficher la nième ligne	awk 'NR == 5' fichier.txt
Nombre de champs (NF)	awk 'NF == 3' fichier.txt



AWK : exercices

```
awk [options] 'conditions {actions}' fichier
```

2

Afficher les objets dont la magnitude est supérieure à 10.01.

3

Afficher les objets qui ont un nom usuel (champ 3 non vide). Afficher uniquement ce nom usuel et la constellation.



Les actions dans la commande awk (1/2)

La partie action spécifie les opérations à exécuter sur chaque enregistrement (ligne) qui satisfait la condition, ou sur chaque enregistrement si aucune condition n'est définie.

awk [options] 'conditions {actions}' fichier

Type d'action	Syntaxe	Description
Imprimer des colonnes	awk '{print \$1, \$3}' fichier.txt	Affiche les colonnes 1 et 3 de chaque ligne.
Imprimer toute la ligne	awk '{print \$0}' fichier.txt	Affiche la ligne entière.
Utilisation de variables	awk '{somme = \$2 + \$3; print somme}' fichier.txt	Calcule la somme des colonnes 2 et 3.
Accumuler une valeur	awk '{total += \$2} END {print "Somme totale:", total}' fichier.txt	Calcule la somme des valeurs de la colonne 2.
Compter des occurrences	awk '\$2 > 20 {count++} END {print count}' fichier.txt	Compte le nombre de lignes où la colonne 2 est > 20.
Modifier un champ	awk '{\$2 = \$2 * 2; print \$0}' fichier.txt	Multiplie la colonne 2 par 2 dans chaque ligne.



Les actions dans la commande awk (2/2)

awk [options] 'conditions {actions}' fichier

Type d'action	Syntaxe	Description
Affichage formaté avec printf	awk '{printf "Nom: %s, Âge: %d\n", \$1, \$2}' fichier.txt	Affiche la colonne 1 comme un nom et la colonne 2 comme un âge avec format.
Instruction conditionnelle if	awk '{ if (\$3 > 80) {print \$1 " : Excellent"} else if (\$3 > 50) {print \$1 " : Moyen"} else {print \$1 " : Faible"} }' fichier.txt	Affiche un message selon la valeur de la colonne 3.
Boucle for	awk '{for (i=1; i<=NF; i++) print \$i}' fichier.txt	Parcourt et affiche chaque champ de la ligne actuelle.
Instructions BEGIN et END	awk 'BEGIN {print "Début du calcul"} {somme += \$2} END {print "Somme totale:", somme}' fichier.txt	Exécute des actions avant ou après le traitement des lignes.



Exercice

1. Calculez et affichez la note moyenne de la classe.

Nom	Mention	Résultat
Alice	Très bien	17
Bob	Bien	15
Bobb	Très bien	16
David	Passable	10
Emma	Insuffisant	8

Pensez à utiliser NR pour compter les lignes, et une instruction END pour afficher le résultat final une seule fois, après avoir parcouru tout le fichier.

2. Écrire un script awk qui recherche et affiche l'objet messier le plus brillant (magnitude la plus petite).

Pour cela, il ne faut pas compter les objets, mais mémoriser celui dont le champ n°2 est plus petit que ce qu'on avait trouvé auparavant — et donc initialiser la variable au plus grand nombre possible.



Les tableaux dans awk

Les tableaux sont des structures utilisées pour stocker des ensembles de données indexées. En awk, ils sont dynamiques et peuvent être indexés numériquement ou par des chaînes, ce qui les rend très flexibles.

Tableaux numériques : les indices sont des entiers.

Syntax

```
awk -F',' 'BEGIN {
    indice = 1
}
{
    Tableau[indice] = $4
    indice++
}
END {
    for (i in Tableau) {
        print Tableau[i]
    }
}' fichier
```

Example

```
awk -F, '{Tableau[NR] = $4} END {for (indice in Tableau) {print Tableau[indice]}}' fichier/messier
```



Dictionnaires (tableaux associatifs)

Les dictionnaires en awk sont simplement des tableaux associatifs où les indices peuvent être des chaînes de caractères. Essayez d'exécuter et de comprendre chacune de ces syntaxes.

Syntaxe 1

```
awk -F'séparateur' '{
    Dictionnaire[clé] = valeur
}
END {
    for (clé in Dictionnaire) {
        print clé,
        Dictionnaire[clé]
    }
}' fichier
```

Example 1

```
awk -F, '($3 != "") { Messier[$1] = $3
} END { for (objet in Messier) { print
objet " : " Messier[objet] } }'
Desktop/TP3/messier
```

```
awk 'BEGIN {
    FS = "séparateur"
}
{
    if (clé in Dictionnaire) {
        print "clé existe déjà"
    }
    else {
        Dictionnaire[clé] = valeur
    }
}
END {
    for (clé in Dictionnaire) {
        print clé, Dictionnaire[clé]
    }
}' fichier
```

Example 2 `awk 'BEGIN { FS = "," } { if ($1 in Dictionnaire) {print "objet existe déjà"} else {Dictionnaire[$1] = "numéro-objet-inséré"}} END { for (n in Dictionnaire) { print n, "", Dictionnaire[n] } }' Desktop/TP3/messier`



Exercice — Question 6

Q6

Écrire un script Awk qui utilise un tableau pour afficher les types d'objets qui existent (galaxie elliptique, galaxie irrégulière, amas globulaire...) : le tableau doit mémoriser les différents champs n°4 du fichier.

Le modifier pour qu'il compte les types (12 différents) : au lieu de seulement mettre « numéro-objet-inséré » dans les cases du tableau, il faut incrémenter un compteur.



La commande find

find : recherche des fichiers dans un répertoire et ses sous-répertoires en fonction de critères comme le nom, la date de modification, la taille, etc.

Syntaxe de la commande

```
find [chemin] [options] [expression] [action]
```



Les options de la commande find (1/2)

`find [chemin] [options] [expression] [action]`

Option	Description
<code>-name "pattern"</code>	Rechercher des fichiers ou dossiers dont le nom correspond au motif donné.
<code>-iname "pattern"</code>	Comme <code>-name</code> mais ignore la casse (majuscule/minuscule).
<code>-type f</code>	Rechercher uniquement des fichiers.
<code>-type d</code>	Rechercher uniquement des répertoires.
<code>-size n</code>	Rechercher des fichiers de taille n (en octets c, kilo-octets k, mégaoctets M).
<code>-mtime n</code>	Rechercher des fichiers modifiés il y a n jours (+n pour plus de n jours, -n pour moins de n jours).
<code>-atime n</code>	Rechercher des fichiers accédés il y a n jours.
<code>-ctime n</code>	Rechercher des fichiers dont le statut a changé il y a n jours.



Les options de la commande find (2/2)

`find [chemin] [options] [expression] [action]`

Option	Description
-perm mode	Rechercher des fichiers avec des permissions spécifiques (ex. 644, +x pour exécutable).
-user name	Rechercher des fichiers appartenant à un utilisateur spécifique.
-group name	Rechercher des fichiers appartenant à un groupe spécifique.
-empty	Rechercher des fichiers ou dossiers vides.
-maxdepth n	Limiter la profondeur de recherche à n niveaux de sous-répertoires.
-mindepth n	Ne commencer la recherche qu'à partir du niveau n de sous-répertoires.
-prune	Exclure des répertoires spécifiques de la recherche.
-regex pattern	Rechercher des fichiers correspondant à une expression régulière donnée.
-iregex pattern	Comme -regex mais ignore la casse.



Les actions dans la commande find

`find [chemin] [options] [expression] [action]`

Action	Description
-print	Afficher le nom des fichiers trouvés (action par défaut).
-delete	Supprimer les fichiers trouvés.
-exec command {} \;	Exécuter une commande sur chaque fichier trouvé ({} est remplacé par le fichier trouvé).
-exec command {} +	Comme -exec, mais exécute la commande sur plusieurs fichiers en une seule fois.
-ok command {} \;	Comme -exec, mais demande confirmation avant d'exécuter la commande sur chaque fichier.
-ls	Afficher les fichiers trouvés avec les détails (comme ls -l).
-fls file	Enregistrer la liste des fichiers trouvés dans un fichier avec les détails.